Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ПРАКТИЧЕСКАЯ РАБОТА 4

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Docker compose

Обучающийся Пухарев Сергей Валерьевич

Факультет Факультет прикладной информатики

Группа K3320

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Пухарев С.В.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

2026

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ 3

1 ПОДГОТОВКА ПРОЕКТА ДЛЯ DOCKER COMPOSE 4

2 ДОБАВЛЕНИЕ REDIS ДЛЯ КЭШИРОВАНИЯ ПОГОДЫ .. 7

3 МАСШТАБИРОВАНИЕ СЕРВИСА 10

4 ФИНАЛЬНЫЙ АРТЕФАКТ 13

ЗАКЛЮЧЕНИЕ 14

ВВЕДЕНИЕ

Цель работы – освоить оркестрацию многоконтейнерного стека (приложение, PostgreSQL, Redis) с помощью Docker Compose, настроить кэширование данных в Redis и реализовать горизонтальное масштабирование сервиса приложения.

Таблица 1 — Мои значения для лабораторной

Номер ИСУ	467203
Персональный порт (App)	5433
Персональный порт (Redis)	6373
Персональный порт (Assistant)	8003
Персональный префикс	pukharev
Секретный ключ	secret03pukharev
Имя базы данных	pukharev03

1 ПОДГОТОВКА ПРОЕКТА ДЛЯ DOCKER COMPOSE

На рисунке 1 представлен файл .env.

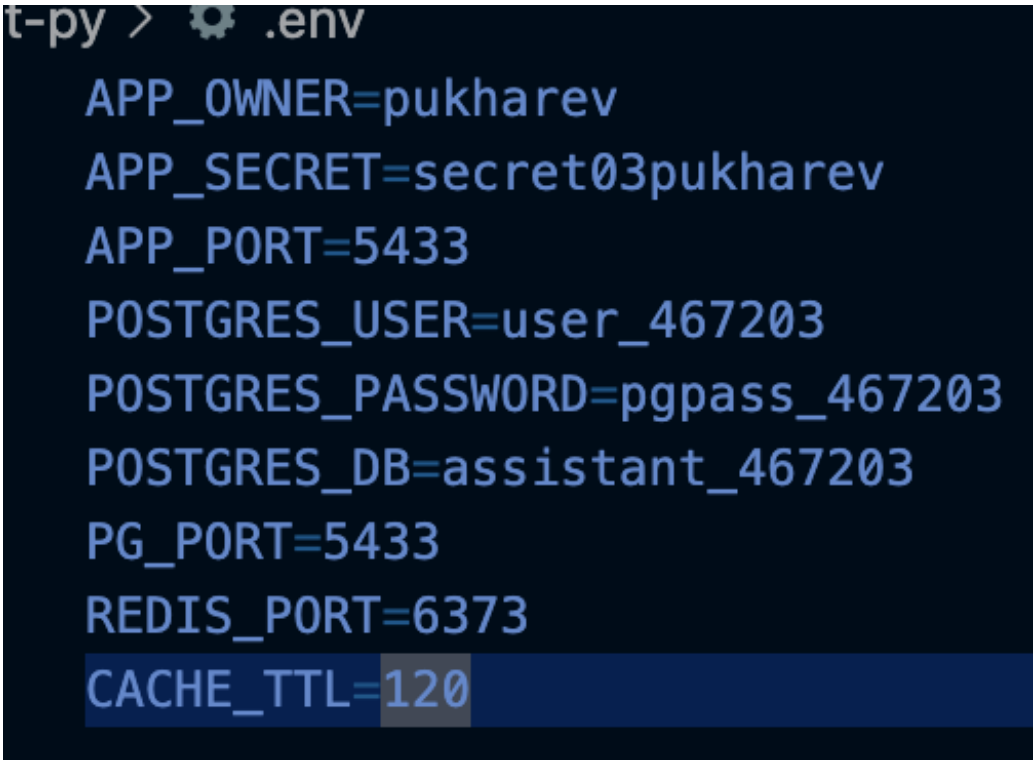


Рисунок 1 — .env

На рисунке 2 представлена первая версия файла docker-compose.yml.

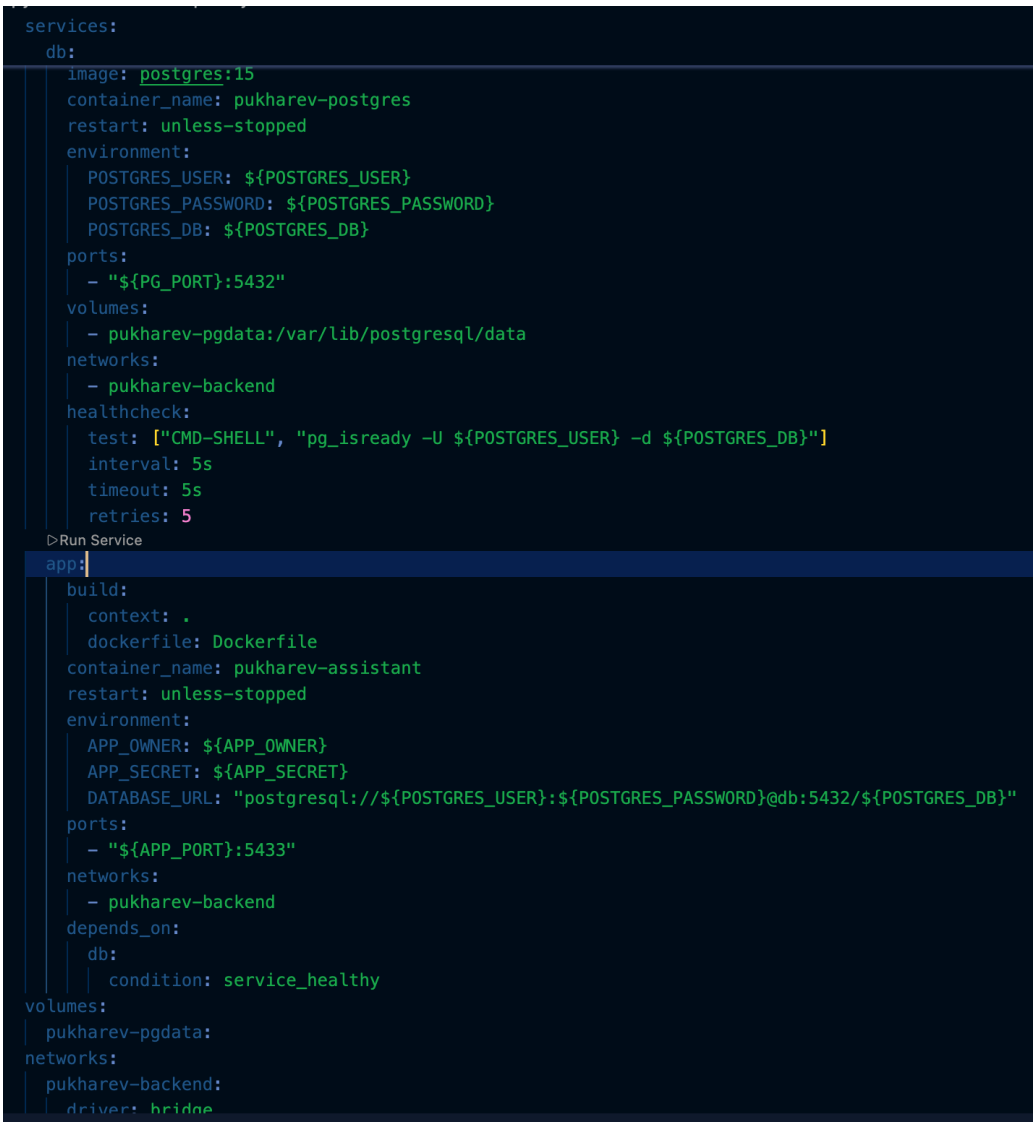


Рисунок 2 — docker-compose.yml

На рисунке 3 представлена проверка логов после запуска.

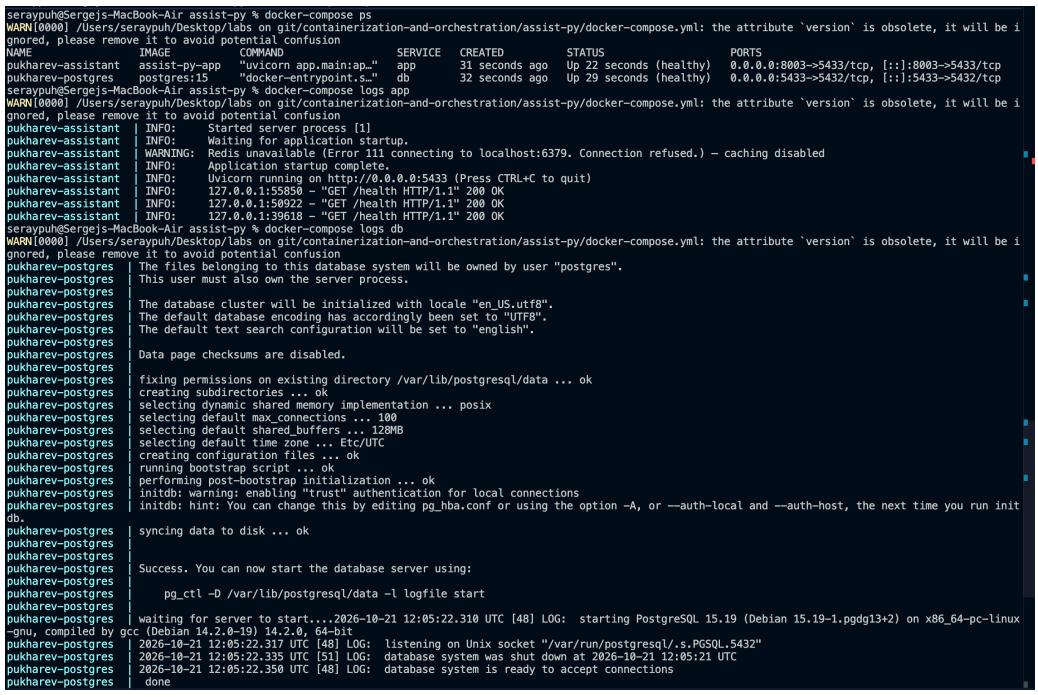


Рисунок 3 — Логи

На рисунке 4 представлено доказательство того, что приложение работает.



Рисунок 4 — curl

2 ДОБАВЛЕНИЕ REDIS ДЛЯ КЭШИРОВАНИЯ ПОГОДЫ

На рисунке 5 представлено добавление Redis в docker-compose.



Рисунок 5 — Redis

Также в app была добавлена зависимость от Redis:

```
      redis:
        condition: service_healthy
```

И переменная REDIS_URL:

```
REDIS_URL: "redis://redis:6379/0"
```

На рисунке 6 представлена проверка подключения Redis и проверка его работоспособности.

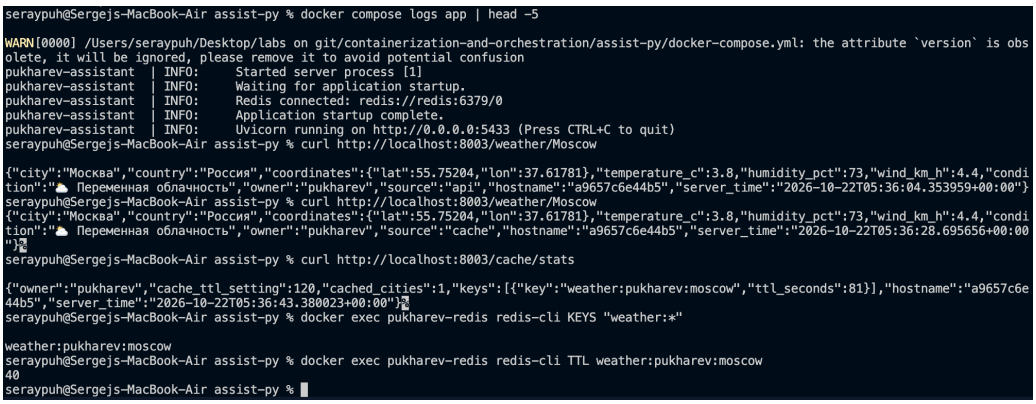


Рисунок 6 — Redis работает

На рисунке 7 представлена проверка сетей и волюмов.



Рисунок 7 — Проверка

3 МАСШТАБИРОВАНИЕ СЕРВИСА

Был создан docker-compose.scale.yml:

```
# docker-compose.scale.yml
services:
  db:
    image: postgres:15
    container_name: pukharev-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pukharev-pgdata:/var/lib/postgresql/data
    networks:
      - pukharev-backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: pukharev-redis
    networks:
      - pukharev-backend
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "5433"
    environment:
      APP_OWNER: ${APP_OWNER}
      APP_SECRET: ${APP_SECRET}
```

```
      DATABASE_URL: postgresql://${POSTGRES_USER}:${
   POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      REDIS_URL: redis://redis:6379/0
      CACHE_TTL: ${CACHE_TTL}
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - pukharev-backend

volumes:
  pukharev-pgdata:

networks:
  pukharev-backend:
    driver: bridge
```

На рисунке 8 представлен запуск и проверка масштабированного контейнера app до трёх.

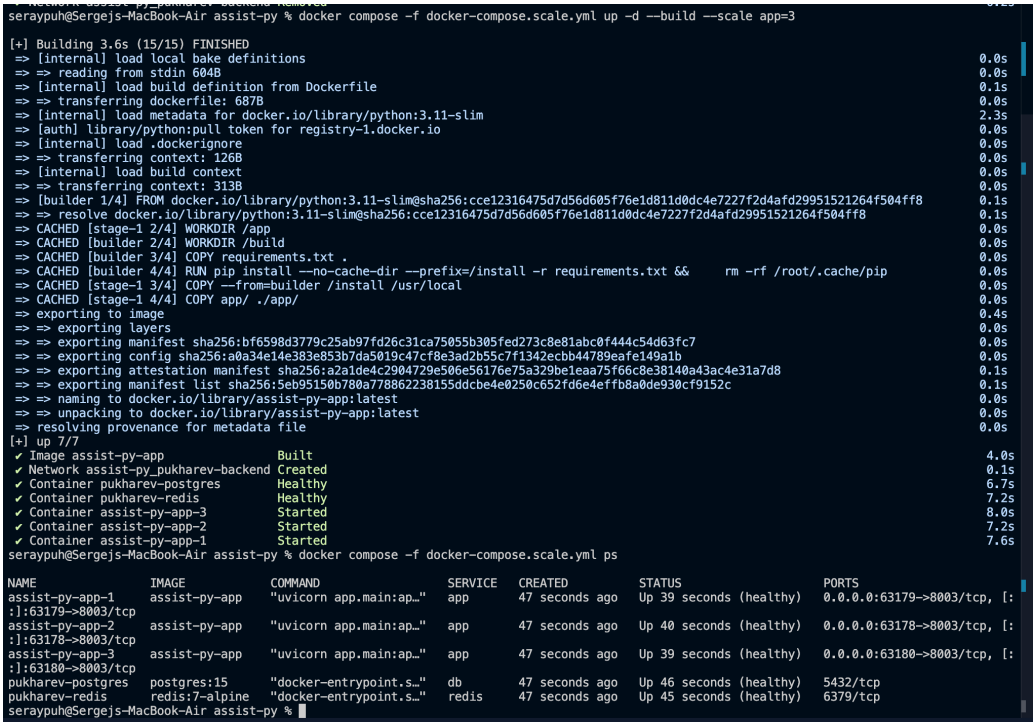


Рисунок 8 — Масштабирование

На рисунке 9 представлена проверка работоспособности и доказательство того, что кеш у приложений общий.

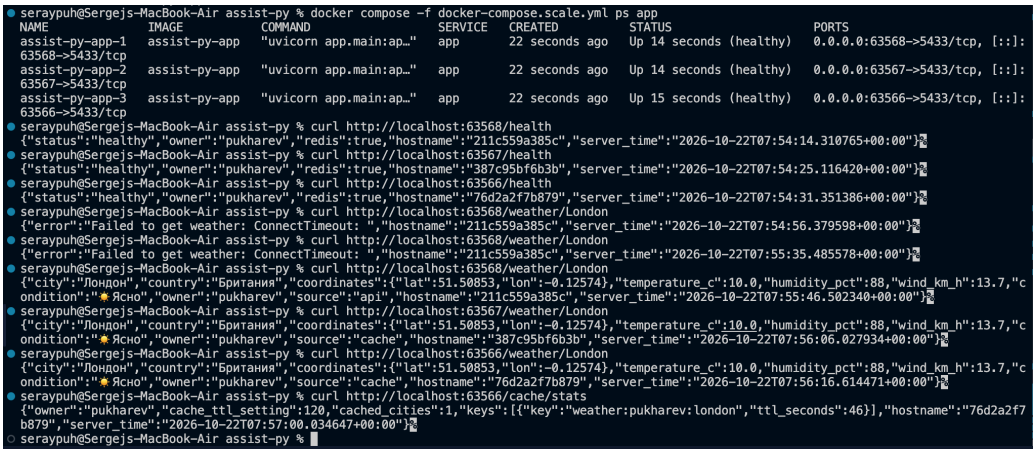


Рисунок 9 — Проверка

4 ФИНАЛЬНЫЙ АРТЕФАКТ

На рисунке 10 представлен финальный артефакт работы.

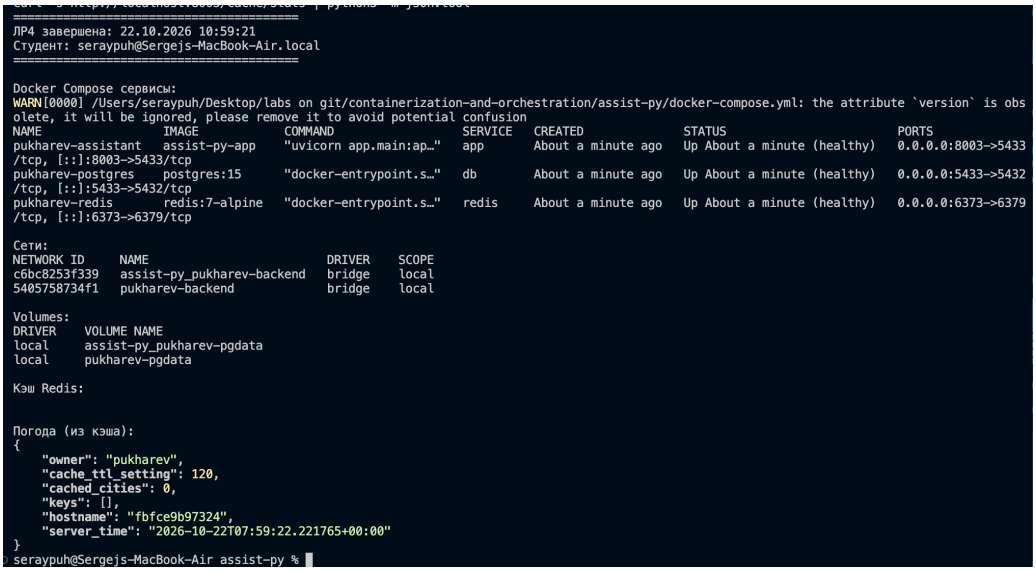


Рисунок 10 — Финальный артефакт

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы был успешно развернут многоконтейнерный стек (веб-приложение, PostgreSQL, Redis) с помощью Docker Compose в единой изолированной сети с сохранением данных в томе, настроен строгий порядок запуска сервисов через healthcheck и depends_on, реализовано эффективное кэширование ответов внешнего API в Redis, а также на практике отработано горизонтальное масштабирование приложения до трёх параллельных экземпляров с общим доступом к кэшу.